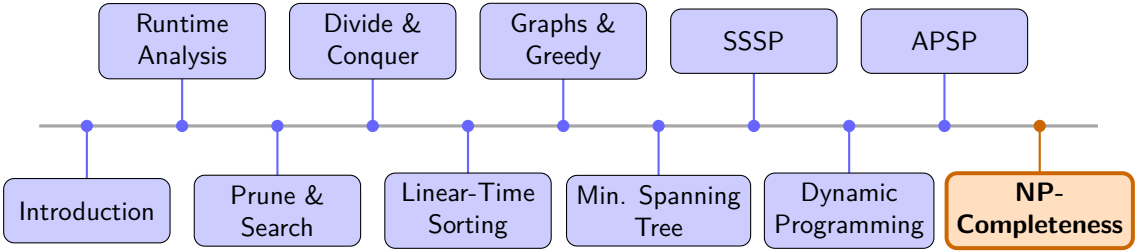
Algorithms

Dr. Mudassir Shabbir

LUMS University

April 8, 2026

Recap & What's Next



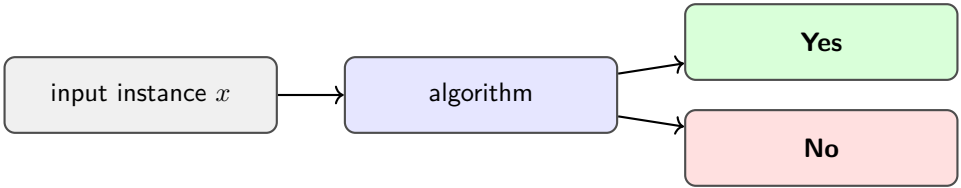
NP-Completeness

Lecture 2: Reductions and Proving Hardness

Recall: Decision Problems

Decision Problem

A **decision problem** takes an input instance and asks a question with answer **Yes** or **No**.



Examples:

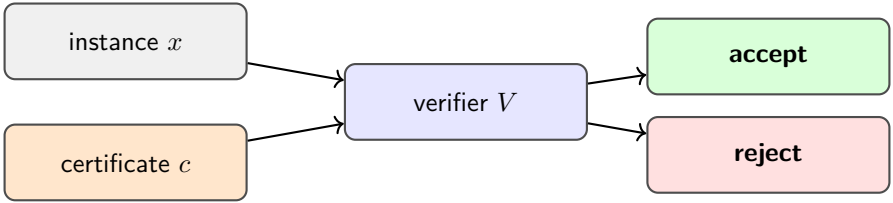
- Is there a path of length $\leq k$ from s to t? (Shortest Path)
- Does graph G have a clique of size $\geq k$? (Clique)
- Is there a tour of cost $\leq B$? (TSP)

We focus on decision problems because complexity classes (P, NP) are cleanly defined.

Recall: Certificates and Verification

Certificate

A **certificate** (or *witness*) is a short piece of extra information that helps prove the answer is **Yes**.



Key rules:

- Certificate has **polynomial size** in $|x|$.
- Answer Yes $\Rightarrow$ *some* c makes V accept.
- Answer No $\Rightarrow$ *no* c can make V accept.

Recall: Class P

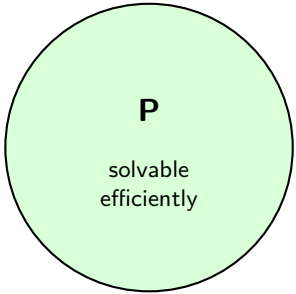
Definition

P is the class of decision problems solvable by an algorithm running in **polynomial time**.

Polynomial time: $O(n^k)$ for some constant k, where n is the input size.

Examples in P:

- Shortest path (Dijkstra, BFS)
- Minimum spanning tree (Kruskal, Prim)
- Maximum flow (Ford-Fulkerson)
- Sorting, matrix multiplication



Recall: Class NP

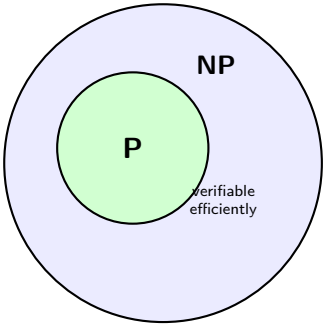
Definition

NP is the class of decision problems for which every **Yes**-instance has a certificate verifiable in **polynomial time**.

Examples in NP:

Problem	Certificate
Hamiltonian Cycle	the cycle itself
Clique	the clique vertices
3-Coloring	the coloring
TSP	the tour

$P \subseteq NP$ means if you can solve it, you can verify it.



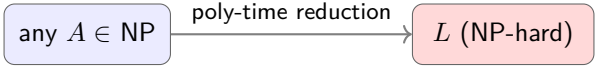
Is $P = NP$?

Nobody knows.

Introducing NP-Hard

Definition (NP-Hard)

A problem L is **NP-hard** if every problem in NP can be reduced to L in polynomial time.



Intuition: L is at least as hard as *every* problem in NP.

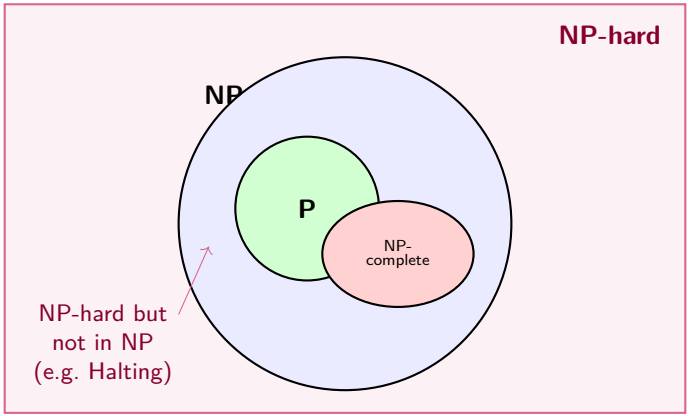
NP-Hard does NOT require $L \in$ NP

An NP-hard problem may be *harder* than NP (e.g. the Halting Problem is NP-hard but not in NP).

NP-Complete = NP-Hard ∩ NP

A problem is **NP-complete** if it is both NP-hard *and* in NP.

The Complexity Landscape



Where We Left Off

NP-Complete (informal)

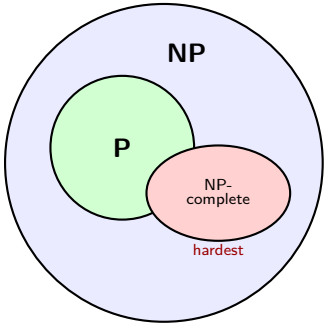
A problem is **NP-complete** if it is in NP *and* is at least as hard as every problem in NP.

We showed:

- **P** $\subseteq$ **NP** (solve $\Rightarrow$ verify)
- NP: yes-answers have short, checkable proofs
- The great question: **P** $\stackrel{?}{=}$ **NP**

Today's challenge

How do we *prove* a problem is NP-complete?



The Challenge of Proving Hardness

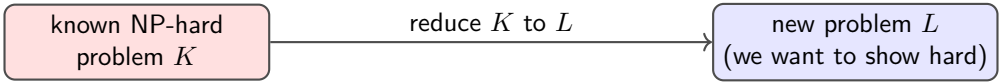
We want to show a problem L is NP-hard. The definition requires:

$$\text{every problem } A \in \text{NP} \text{ reduces to } L$$

That sounds impossible!

There are infinitely many problems in NP. We cannot check them one by one.

However: we do not need to re-prove hardness from scratch each time.



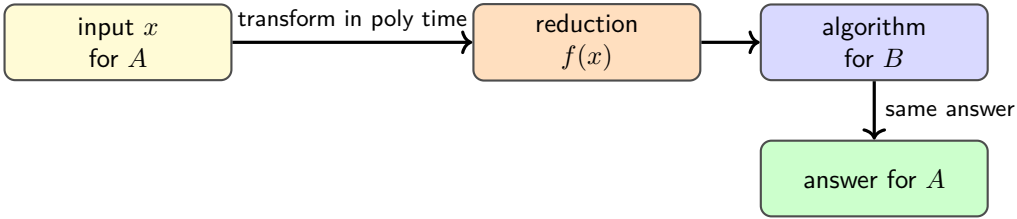
If we can solve L efficiently, we can solve K efficiently: contradiction!

Polynomial-Time Reductions

Definition

A **polynomial-time reduction** from problem A to problem B, written $A \leq_p B$, is a function f such that:

1. f is computable in polynomial time,
2. for every input x: $x \in A \iff f(x) \in B$.

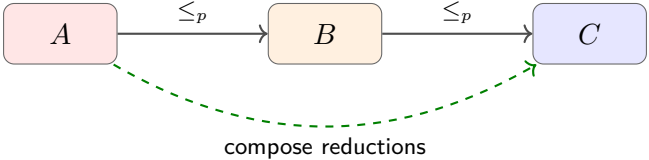


Key point: An algorithm for B gives us an algorithm for A for *free*.

Insights Regarding Reductions

Transitivity

If $A \leq_p B$ and $B \leq_p C$, then $A \leq_p C$.



Hardness propagates upward

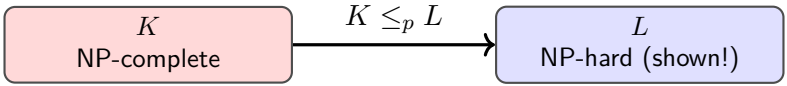
If $A \leq_p B$ and $B \in \mathsf{P}$, then $A \in \mathsf{P}$.

Contrapositive (most useful in practice)

If $A \leq_p B$ and $A \notin \mathsf{P}$, then $B \notin \mathsf{P}$.

What Reductions Imply for NP-Completeness

Suppose K is a **known NP-complete** problem and we show $K \leq_p L$.



Conclusion: L is NP-hard (every NP problem reduces to L via transitivity).

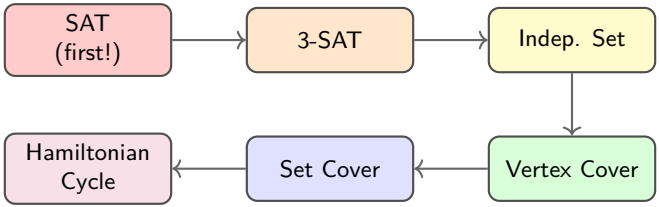
Two-Step Proof Recipe for NP-Completeness

To prove L is NP-complete:

1. **Show $L \in$ NP.** Describe a polynomial-time verifier.
2. **Show L is NP-hard.** Pick a known NP-complete K and give a poly-time reduction $K \leq_p L$.

The Domino Effect

Once the **first** NP-complete problem is established, the rest fall like dominoes.



We/You will prove several of these arrows.

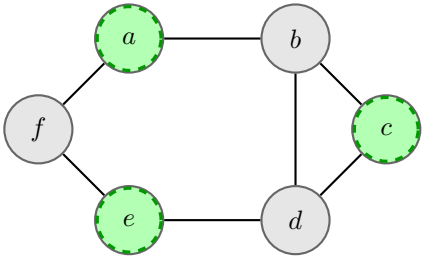
Independent Set

Definition

Given a graph $G = (V, E)$ and integer k.
Independent Set (IS): Does there exist a set $S \subseteq V$ with $|S| \geq k$ such that *no two vertices in S are adjacent*?

Example — $k = 3$: ✓Yes

The green vertices $\{a, c, e\}$ form an independent set: no edge between any two of them.



Vertex Cover

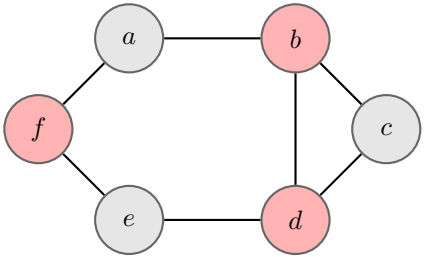
Definition

Given a graph $G = (V, E)$ and integer k.

Vertex Cover (VC): Does there exist a set $S \subseteq V$ with $|S| \leq k$ such that *every edge has at least one endpoint in* S?

Example — $k = 3$: ✓Yes

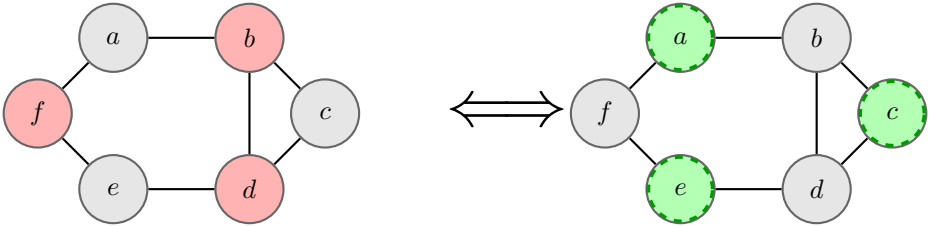
The red vertices $\{b, d, f\}$ cover every edge — each edge touches at least one red vertex.



The Beautiful Complement Relationship

Observation: In the previous example, notice that:

$\{b, d, f\}$ is a vertex cover $\iff$ $\{a, c, e\}$ is an independent set



VC: $\{b, d, f\}$, $k = 3$

IS: $\{a, c, e\}$, $k' = 3$

The complement of a vertex cover is always an independent set, and vice versa!

Key Lemma: VC and IS are Complements

Lemma

$S \subseteq V$ is a vertex cover of G if and only if $V \setminus S$ is an independent set of G.

Proof.

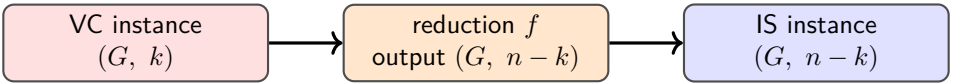
($\Rightarrow$) Suppose S is a vertex cover. Consider any two vertices $u, v \in V \setminus S$. If $\{u, v\}$ were an edge, then neither endpoint would be in S, contradicting that S is a vertex cover. So $V \setminus S$ has no edges: it is an independent set. ✓

($\Leftarrow$) Suppose $V \setminus S$ is an independent set. Consider any edge $\{u, v\} \in E$. Since $V \setminus S$ has no edges, at least one of u, v must lie in S. So every edge is covered by S: it is a vertex cover. ✓

Reduction: Vertex Cover $\leq_p$ Independent Set

The Reduction

Given an instance (G, k) of Vertex Cover, output the instance $(G, n-k)$ of Independent Set, where $n = |V|$.



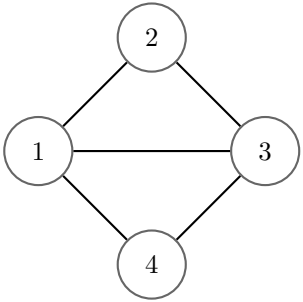
Why does this work? (Correctness)

By the Lemma: a vertex cover S of size $\leq k$ exists $\iff$ an independent set $V \setminus S$ of size $\geq n-k$ exists.

Running time: The reduction just outputs the same graph with $n-k$ instead of k. This takes $O(1)$ time — definitely polynomial. ✓

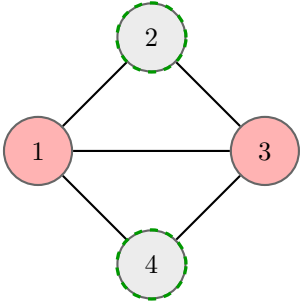
Example: VC $\rightarrow$ IS Reduction

VC Input: $k = 2$



Is there a VC of size ≤ 2?

IS Output: $k' = n - k = 2$



Red = VC $\{1, 3\}$; Dashed = IS $\{2, 4\}$. Complementary!

Same graph, $k' = n - k$. The reduction is a **one-liner**!

IS is NP-Complete

Theorem

Independent Set is NP-complete.

Step 1 — IS $\in$ NP.
Certificate: the set S itself. Check $|S| \geq k$ and verify no two vertices share an edge. Time: $O(n^2)$. ✓

Step 2 — IS is NP-hard.
Vertex Cover $\leq_p$ IS (the complement reduction above). Since VC is NP-complete, IS inherits NP-hardness. ✓

$$\text{VC} \leq_p \text{IS} \quad \Longrightarrow \quad \text{IS is NP-hard}$$